```
M219 R123.45 (orients the live tool to 123.45 degrees) ;
```

M133/M134/M135 Live Tool Fwd/Rev/Stop (Optional)

Refer to page **361** for a complete description of these M-codes.

6.7 Macros (Optional)

6.7.1 Macros Introduction

NOTE: *This control feature is optional; call your HFO for information on how to purchase it.*

Macros add capabilities and flexibility to the control that are not possible with standard G-code. Some possible uses are: families of parts, custom canned cycles, complex motions, and driving optional devices. The possibilities are almost endless.

A Macro is any routine/subprogram that you can run multiple times. A macro statement can assign a value to a variable, read a value from a variable, evaluate an expression, conditionally or unconditionally branch to another point within a program, or conditionally repeat some section of a program.

Here are a few examples of the applications for Macros. The examples are outlines and not complete macro programs.

Useful G and M Codes

`M00`, `M01`, `M30` - Stop Program

`G04` - Dwell

`G65 Pxx` - Macro subprogram call. Allows passing of variables.

`M129` - Set output relay with M-FIN.

`M59` - Set output relay.

`M69` - Clear output relay.

`M96 Pxx Qxx` - Conditional Local Branch when Discrete Input Signal is 0

`M97 Pxx` - Local Sub Routine Call

`M98 Pxx` - Sub Program Call

`M99` - Sub Program Return or Loop

`G103` - Block Lookahead Limit. No cutter comp allowed.

`M109` - Interactive User Input (refer to page **357**)

Round Off

The control stores decimal numbers as binary values. As a result, numbers stored in variables can be off by 1 least significant digit. For example, the number 7 stored in macro variable `#10000`, may later be read as 7.000001, 7.000000, or 6.999999. If your statement was

```
IF [#10000 EQ 7]… ;
```

it may give a false reading. A safer way of programming this would be

```
IF [ROUND [#10000] EQ 7]… ;
```

This issue is usually a problem only when you store integers in macro variables where you do not expect to see a fractional part later.

Look-ahead

Look-ahead is a very important concept in macro programming. The control attempts to process as many lines as possible ahead of time in order to speed up processing. This includes the interpretation of macro variables. For example,

```
#12012 = 1 ;
G04 P1. ;
#12012 = 0 ;
```

This is intended to turn an output on, wait 1 second, and then turn it off. However, lookahead causes the output to turn on then immediately back off while the control processes the dwell. `G103 P1` is used to limit lookahead to 1 block. To make this example work properly, modify it as follows:

```
G103 P1 (See the G-code section of the manual for a further
explanation of G103) ;
;
#12012=1 ;
G04 P1. ;
;
```

```
;
;
#12012=0 ;
```

Block Look-Ahead and Block Delete

The Haas control uses block look-ahead to read and prepare for blocks of code that come after the current block of code. This lets the control transition smoothly from one motion to the next. `G103` limits how far ahead the control looks at blocks of code. The P`nn`address code in `G103` specifies how far ahead the control is allowed to look. For additional information, refer to `G103` on page **318**.

Block Delete mode lets you selectively skip blocks of code. Use a `/` character at the beginning of the program blocks that you want to skip. Press **[BLOCK DELETE]** to enter the Block Delete mode. While Block Delete mode is active, the control does not execute the blocks marked with a `/` character. For example:

Using a

```
/M99 (Sub-Program Return) ;
```

before a block with

```
M30 (Program End and Rewind) ;
```

makes the sub-program a main program when **[BLOCK DELETE]** is on. The program is used as a sub-program when Block Delete is off.

When a block delete token "/" is used, even if Block Delete mode is not active, the line will block look ahead. This is useful for debugging macro processing within NC programs.

6.7.2 Operation Notes

You save or load macro variables through the Net Share or USB port, much like settings, and offsets.

Macro Variable Display Page

The local and global macro variables `#1` - `#33` and `#10000` - `#10999` are displayed and modified through the Current Commands display.

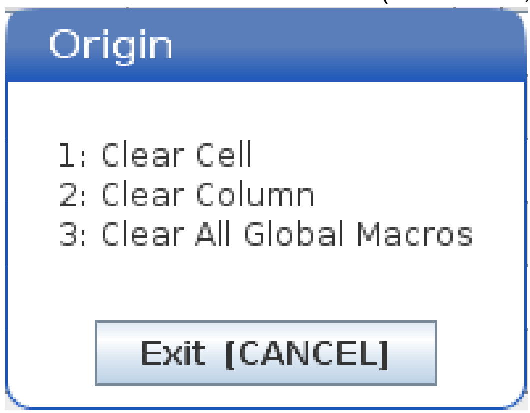
NOTE: *Internal to the machine, 10000 is added to 3-digit macro variables. For example: Macro 100 is displayed as 10100.*

1. Press **[CURRENT COMMANDS]** and use navigation keys to get to the `Macro Vars` page.

   As the control interprets a program, the variable changes and results are displayed on the `Macro Vars` display page.

2. Enter a value (maximum is 999999.000000) and then press **[ENTER]** to set the macro variable. Press **[ORIGIN]** to clear macro variables, this displays the Origin clear entry popup. Press number 1 - 3 to make a selection or press **[CANCEL]** to exit.

F6.9: Origin Clear Entry Popup. `1: Clear Cell` - Clears the highlighted cell to zero. `2: Clear Column` - Clears the active cursor column entries to zero. `3: Clear All Global Macros` - Clears Global Macro entries (Macro 1-33, 10000-10999) to zero.



3. To search for a variable, enter the macro variable number and press the up or down arrow.

4. The variables displayed represent values of the variables when the program runs. At times, this may be up to 15 blocks ahead of actual machine actions. Debugging programs is easier when a G103 P1 is inserted at the beginning of a program to limit block buffering. A G103 without the P value can be added after the macro variable blocks in the program. For a macro program to operate correctly it is recommended that the `G103 P1` be left in the program during loading of variables. For more details about G103 see the G-code section of the manual.

Display Macro Variables in the Timers And Counters Window

In the `Timers And Counters` window, you can display the values of any two macro variables and assign them a display name.

To set which two macro variables display in the `Timers And Counters` window:

1. Press **[CURRENT COMMANDS]**.
2. Use the navigation keys to select the `TIMERS` page.
3. Highlight the `Macro Label #1` name or `Macro Label #2` name.

4. Key in a new name, and press **[ENTER]**.
5. Use arrow keys to pick the **`Macro Assign #1`** or **`Macro Assign #2`** entry field (corresponding to your chosen **`Macro Label`** name).
6. Key in the macro variable number (without #) and press **[ENTER]**.

On the **`Timers And Counters`** window, the field to the right of the entered **`Macro Label`** (#1 or #2) name displays the assigned variable value.

Macro Arguments

The arguments in a `G65` statement are a means to send values to a macro subprogram and set the local variables of a macro subprogram.

The next (2) tables indicate the mapping of the alphabetic address variables to the numeric variables used in a macro subprogram.

Alphabetic Addressing

T6.1: Alphabetic Address Table

Address	Variable	Address	Variable
A	1	N	-
B	2	O	-
C	3	P	-
D	7	Q	17
E	8	R	18
F	9	S	19
G	-	T	20
H	11	U	21
I	4	V	22
J	5	W	23
K	6	X	24
L	-	Y	25
M	13	Z	26

Alternate Alphabetic Addressing

Address	Variable	Address	Variable	Address	Variable
A	1	K	12	J	23
B	2	I	13	K	24
C	3	J	14	I	25
I	4	K	15	J	26
J	5	I	16	K	27
K	6	J	17	I	28
I	7	K	18	J	29
J	8	I	19	K	30
K	9	J	20	I	31
I	10	K	21	J	32
J	11	I	22	K	33

Arguments accept any floating-point value to four decimal places. If the control is in metric, it will assume thousandths (.000). In example below, local variable #1 will receive .0001. If a decimal is not included in an argument value, such as:

```
G65 P9910 A1 B2 C3 ;
```

The values are passed to macro subprograms according to this table:

Integer Argument Passing (no decimal point)

Address	Variable		Address	Variable		Address	Variable
A	.0001		J	.0001		S	1.
B	.0002		K	.0001		T	1.
C	.0003		L	1.		U	.0001

Address	Variable		Address	Variable		Address	Variable
D	1.		M	1.		V	.0001
E	1.		N	-		W	.0001
F	1.		O	-		X	.0001
G	-		P	-		Y	.0001
H	1.		Q	.0001		Z	.0001
I	.0001		R	.0001			

All 33 local macro variables can be assigned values with arguments by using the alternate addressing method. The following example shows how to send two sets of coordinate locations to a macro subprogram. Local variables `#4` through `#9` would be set to .0001 through .0006 respectively.

Example:

```
G65 P2000 I1 J2 K3 I4 J5 K6;
```

The following letters cannot be used to pass parameters to a macro subprogram: G, L, N, O or P.

Macro Variables

There are (3) categories of macro variables: local, global, and system.

Macro constants are floating-point values placed in a macro expression. They can be combined with addresses A-Z, or they can stand alone when used within an expression. Examples of constants are 0.0001, 5.3 or -10.

Local Variables

Local variables range between #1 and #33. A set of local variables is available at all times. When a call to a subprogram with a G65 command is executed, local variables are saved and a new set is available for use. This is called nesting of local variables. During a G65 call, all new local variables are cleared to undefined values and any local variables that have corresponding address variables in the G65 line are set to G65 line values. Below is a table of the local variables along with the address variable arguments that change them:

Variable:	1	2	3	4	5	6	7	8	9	10	11
Address:	A	B	C	I	J	K	D	E	F		H
Alternate:							I	J	K	I	J
Variable:	12	13	14	15	16	17	18	19	20	21	22
Address:		M				Q	R	S	T	U	V
Alternate:	K	I	J	K	I	J	K	I	J	K	I
Variable:	23	24	25	26	27	28	29	30	31	32	33
Address:	W	X	Y	Z							
Alternate:	J	K	I	J	K	I	J	K	I	J	K

Variables 10, 12, 14- 16 and 27- 33 do not have corresponding address arguments. They can be set if a sufficient number of I, J and K arguments are used as indicated above in the section about arguments. Once in the macro subprogram, local variables can be read and modified by referencing variable numbers 1- 33.

When the L argument is used to do multiple repetitions of a macro subprogram, the arguments are set only on the first repetition. This means that if local variables 1- 33 are modified in the first repetition, then the next repetition will have access only to the modified values. Local values are retained from repetition to repetition when the L address is greater than 1.

Calling a subprogram via an M97 or M98 does not nest the local variables. Any local variables referenced in a subprogram called by an M98 are the same variables and values that existed prior to the M97 or M98 call.

Global Variables

Global variables are accessible at all times and remain in memory when power is turned off. There is only one copy of each global variable. Global variables are numbered #10000-#10999. Three legacy ranges: (#100-#199, #500-#699, and #800-#999) are included. The legacy 3 digit macro variables begin at the #10000 range; ie., macro variable #100 is displayed as #10100.

NOTE: *Using variable #100 or #10100 in a program the control will access the same data. Using either variable number is acceptable.*

Sometimes, factory-installed options use global variables, for example, probing and pallet changers, etc. Refer to the Macro Variables Table on page **204** for global variables and their use.

CAUTION: *When you use a global variable, make sure that no other programs on the machine use the same global variable.*

System Variables

System variables let you interact with a variety of control conditions. System variable values can change the function of the control. When a program reads a system variable, it can modify its behavior based on the value in the variable. Some system variables have a Read Only status; this means that you cannot modify them. Refer to the Macro Variables Table on page **204** for a list of system variables and their use.

Macro Variables Table

The macro variables table of local, global, and system variables and their usage follows. The new generation control variables list includes legacy variables.

NGC Variable	Legacy Variable	Usage
#0	#0	Not a number (read only)
#1- #33	#1- #33	Macro call arguments
#10000- #10199	#100- #199	General-purpose variables saved on power off
#10200- #10399	N/A	General-purpose variables saved on power off
#10400- #10499	N/A	General-purpose variables saved on power off

NGC Variable	Legacy Variable	Usage
#10500- #10549	#500-#549	General-purpose variables saved on power off
#10550- #10580	#550-#580	Probe calibration data (if installed)
#10581- #10699	#581- #699	General-purpose variables saved on power off
#10700- #10799	#700- #749	Hidden variables for internal use only
#10709	#709	Used for the Fixture Clamp Input. Do not use for general purpose.
#10800- #10999	#800- #999	General-purpose variables saved on power off
#11000- #11063	N/A	64 discrete inputs (read only)
#1064- #1068	#1064- #1068	Maximum axis loads for X, Y, Z, A, and B Axes, respectively
#1080- #1087	#1080- #1087	Raw analog to digital inputs (read only)
#1090- #1098	#1090- #1098	Filtered analog to digital inputs (read only)
#1098	#1098	Spindle load with Haas vector drive (read only)
#1264- #1268	#1264- #1268	Maximum axis loads for C, U, V, W, and T-axes respectively
#1601- #1800	#1601- #1800	Number of flutes on tools #1 through 200
#1801- #2000	#1801- #2000	Maximum recorded vibrations of tools 1 through 200
#2001- #2050	#2001- #2050	X Axis tool shift offsets
#2051- #2100	#2051- #2100	Y Axis tool shift offsets
#2101- #2150	#2101- #2150	Z Axis tool shift offsets
#2201- #2250	#2201- #2250	Tool nose radius wear offsets
#2301- #2350	#2301- #2350	Tool tip direction
#2701- #2750	#2701- #2750	X Axis tool wear offsets
#2751- #2800	#2751- #2800	Y Axis tool wear offsets
#2801- #2850	#2801- #2850	Z Axis tool wear offsets
#2901- #2950	#2901- #2950	Tool nose radius wear offsets

NGC Variable	Legacy Variable	Usage
#3000	#3000	Programmable alarm
#3001	#3001	Millisecond timer
#3002	#3002	Hour timer
#3003	#3003	Single block suppression
#3004	#3004	Override **[FEED HOLD]** control
#3006	#3006	Programmable stop with message
#3011	#3011	Year, month, day
#3012	#3012	Hour, minute, second
#3020	#3020	Power on timer (read only)
#3021	#3021	Cycle start timer
#3022	#3022	Feed timer
#3023	#3023	Present part timer (read only)
#3024	#3024	Last complete part timer
#3025	#3025	Previous part timer (read only)
#3026	#3026	Tool in spindle (read only)
#3027	#3027	Spindle RPM (read only)
#3030	#3030	Single block
#3032	#3032	Block delete
#3033	#3033	Opt stop
#3196	#3196	Cell safe timer
#3201- #3400	#3201- #3400	Actual diameter for tools 1 through 200
#3401- #3600	#3401- #3600	Programmable coolant positions for tools 1 through 200
#3901#3901	#3901#3901	M30 count 1

NGC Variable	Legacy Variable	Usage
#3902#3902	#3902#3902	M30 count 2
#4001- #4021	#4001- #4021	Previous block G-code group codes
#4101- #4126	#4101- #4126	Previous block address codes. **NOTE:** *(1) Mapping of 4101 to 4126 is the same as the alphabetic addressing of Macro Arguments section; e.g., the statement X1.3 sets variable #4124 to 1.3.*
#5001- #5006	#5001- #5006	Previous block end position
#5021- #5026	#5021- #5026	Present machine coordinate position
#5041- #5046	#5041- #5046	Present work coordinate position
#5061- #5069	#5061- #5069	Present skip signal position - X, Y, Z, A, B, C, U, V, W
#5081- #5086	#5081- #5086	Present tool offset
#5201- #5206	#5201- #5206	G52 work offsets
#5221- #5226	#5221- #5226	G54 work offsets
#5241- #5246	#5241- #5246	G55 work offsets
#5261- #5266	#5261- #5266	G56 work offsets
#5281- #5286	#5281- #5286	G57 work offsets
#5301- #5306	#5301- #5306	G58 work offsets
#5321- #5326	#5321- #5326	G59 work offsets
#5401- #5500	#5401- #5500	Tool feed timers (seconds)
#5501- #5600	#5501- #5600	Total tool timers (seconds)

NGC Variable	Legacy Variable	Usage
#5601- #5699	#5601- #5699	Tool life monitor limit
#5701- #5800	#5701- #5800	Tool life monitor counter
#5801- #5900	#5801- #5900	Tool load monitor maximum load sensed so far
#5901- #6000	#5901- #6000	Tool load monitor limit
#6001- #6999	#6001- #6999	Reserved. Do not use.
#6198	#6198	NGC/CF flag
#7001- #7006	#7001- #7006	G110 (G154 P1) additional work offsets
#7021- #7026	#7021- #7026	G111 (G154 P2) additional work offsets
#7041- #7386	#7041- #7386	G112 - G129 (G154 P3 - P20) additional work offsets
#8500	#8500	Advanced Tool Management (ATM) group ID
#8501	#8501	ATM percent of available tool life of all tools in the group
#8502	#8502	ATM total available tool usage count in the group
#8503	#8503	ATM total available tool hole count in the group
#8504	#8504	ATM total available tool feed time (in seconds) in the group
#8505	#8505	ATM total available tool total time (in seconds) in the group
#8510	#8510	ATM next tool number to be used
#8511	#8511	ATM percent of available tool life of the next tool
#8512	#8512	ATM available usage count of the next tool
#8513	#8513	ATM available hole count of the next tool
#8514	#8514	ATM available feed time of the next tool (in seconds)
#8515	#8515	ATM available total time of the next tool (in seconds)
#8550	#8550	Individual tool ID
#8551	#8551	Number of flutes of tools

NGC Variable	Legacy Variable	Usage
#8552	#8552	Maximum recorded vibrations
#8553	#8553	Tool length offsets
#8554	#8554	Tool length wear
#8555	#8555	Tool diameter offsets
#8556	#8556	Tool diameter wear
#8557	#8557	Actual diameter
#8558	#8558	Programmable coolant position
#8559	#8559	Tool feed timer (seconds)
#8560	#8560	Total tool timers (seconds)
#8561	#8561	Tool life monitor limit
#8562	#8562	Tool life monitor counter
#8563	#8563	Tool load monitor maximum load sensed so far
#8564	#8564	Tool load monitor limit
#9000	#9000	Thermal comp accumulator
#9000- #9015	#9000- #9015	Reserved (duplicate of axis thermal accumulator)
#9016-#9016	#9016-#9016	Thermal spindle comp accumulator
#9016- #9031	#9016- #9031	Reserved (duplicate of axis thermal accumulator from spindle)
#10000- #10999	N/A	General purpose variables
#11000- #11255	N/A	Discrete inputs (read only)
#12000- #12255	N/A	Discrete outputs
#13000- #13063	N/A	Filtered analog to digital inputs (read only)
#13013	N/A	Coolant level
#14001- #14006	N/A	G110(G154 P1) additional work offsets